

CHAPTER 1 : INTRODUCTION

In today's rapidly evolving digital landscape, virtual collaboration and remote interaction have become integral to modern communication. Traditional communication tools such as video conferencing and chat platforms lack spatial awareness, leading to less immersive and engaging experiences. Platforms like Gather and Zep have introduced a new paradigm by enabling users to interact within virtual spaces based on proximity.

Recognizing the potential of spatial computing and real-time systems, the project titled **"2D Real-Time Metaverse Platform"** was developed to create an interactive virtual environment where users can move, interact, and communicate dynamically. The system leverages real-time communication using WebSockets, allowing seamless synchronization of user movement and interactions across multiple clients.

The project introduces a structured environment consisting of organizations, spaces, rooms, and maps. Each map represents a 2D grid-based world populated with interactive elements and users represented as avatars. The movement of users is handled in real time, and interactions are triggered based on spatial proximity, enabling a more natural and intuitive communication model.

To ensure scalability and efficiency, the system is built using modern technologies such as Next.js for frontend rendering, Phaser for 2D visualization, WebSocket for real-time communication, and PostgreSQL with Prisma for data management.

In summary, this project bridges the gap between static communication platforms and immersive virtual environments by introducing a scalable, real-time spatial interaction system that enhances user engagement and collaboration.

1.1 NEED AND OBJECTIVE

In the current era of remote work and online collaboration, communication tools often lack context and spatial interaction. Users interact through text or video without any sense of presence or proximity, which limits engagement and natural communication.

Existing platforms fail to replicate real-world interaction dynamics where communication depends on physical closeness. For example, in an office or social environment, conversations occur naturally when individuals are nearby. However, in digital systems, this behavior is not represented effectively.

Additionally, current systems do not provide real-time synchronization of user movement or environment-based interaction, leading to disconnected and less immersive experiences. This highlights the need for a system that can:

- Simulate real-world spatial interaction
- Enable real-time communication and movement
- Provide structured virtual environments

- Support scalable multi-user interaction

Objectives of the Project

This project aims to develop a real-time spatial metaverse platform that enables interactive virtual collaboration. The primary objectives are:

1. To design and implement a structured virtual environment consisting of organizations, spaces, rooms, and maps.
2. To enable real-time user movement and synchronization using WebSocket-based communication.
3. To implement proximity-based interaction where communication is triggered based on distance between users.
4. To build a scalable backend system for managing users, spaces, and real-time events.
5. To develop an interactive frontend interface for rendering maps and user avatars using a game engine.

1.2 LITERATURE STUDY

The concept of virtual environments and spatial interaction has gained significant attention in recent years, particularly with the rise of metaverse platforms and real-time collaboration tools.

Research in real-time systems emphasizes the importance of low-latency communication protocols such as WebSockets for enabling continuous bidirectional data transfer between clients and servers. These systems are widely used in multiplayer games and collaborative platforms.

Studies in spatial computing highlight the importance of proximity-based interaction, where user behavior and communication depend on their relative positions within a virtual space. This approach has been successfully implemented in platforms like Gather, where users can engage in conversations only when they are within a certain distance.

Additionally, modern frontend frameworks like Next.js combined with game engines such as Phaser have enabled developers to build highly interactive and visually engaging applications. These tools allow efficient rendering of dynamic environments and real-time updates.

The integration of database management systems such as PostgreSQL with ORMs like Prisma ensures efficient data storage, retrieval, and scalability, making them suitable for real-time applications.

Overall, existing research supports the development of systems that combine real-time communication, spatial interaction, and scalable architecture to create immersive digital experiences.

1.3 METHODOLOGY

The proposed system follows a modular and structured approach to design and implementation. The overall workflow consists of multiple stages, including data management, real-time communication, and frontend rendering.

1. System Design

The system is divided into the following core components:

- User Management
- Space and Room Management
- Map and Element System
- Real-Time Communication Engine
- Proximity Detection System

2. Real-Time Communication

WebSocket connections are established between the client and server to enable real-time communication. Each user is assigned a session ID upon connection, and all interactions are handled through event-based messaging.

3. User Movement System

User movement is handled through continuous updates of position coordinates (x, y). These updates are sent to the server and broadcasted to all users in the same space.

4. Proximity Detection

A distance-based algorithm is used to detect nearby users. When users enter or leave a predefined radius, events such as:

- PROXIMITY_ENTER
- PROXIMITY_LEAVE

are triggered to manage interactions.

5. Data Management

The system uses PostgreSQL for storing structured data such as users, spaces, maps, and elements. Prisma ORM is used to define schema models and manage database operations efficiently.

6. Frontend Rendering

The frontend is built using Next.js and Phaser, where:

- Maps are rendered as 2D grids
- Users are displayed as avatars
- Movement and interactions are visually updated in real time

1.4 SOFTWARES USED

The following tools and technologies were used in the development of this project:

1. Next.js

Used for building the frontend interface and handling routing and server-side rendering.

2. Phaser

A 2D game engine used to render maps, avatars, and animations in real time.

3. WebSocket

Enables real-time bidirectional communication between server and clients.

4. Prisma

Used as an ORM to interact with the PostgreSQL database.

5. PostgreSQL

Used for storing structured data related to users, spaces, maps, and interactions.

6. Docker

Used for containerization and deployment of the database and backend services.

7. Visual Studio Code

Primary development environment for writing and debugging code.

1.5 WORK BREAKDOWN STRUCTURE

The project is divided into the following major components:

1. Backend Module

- Handles API requests
- Manages database operations
- Controls business logic

2. Real-Time Engine Module

- WebSocket server
- User session handling
- Movement broadcasting
- Proximity detection

3. Database Module

- Schema design using Prisma
- Storage of users, spaces, maps, and elements

4. Frontend Module

- Rendering maps using Phaser
- Handling user input and movement
- Displaying real-time updates

5. Interaction Module

- Proximity-based communication
- Event handling (join, move, leave)

CHAPTER 2 : IMPLEMENTATION

1. Real-Time Server > server.ts

The core of the system is built around a WebSocket-based real-time communication server. This server is responsible for managing user connections, handling events, and synchronizing state across all connected clients.

The server is implemented using Node.js and the WebSocket protocol, enabling bidirectional communication between the client and server.

Each user establishes a WebSocket connection and is assigned a unique session. The server maintains in-memory data structures to track:

- Active users
- User positions (x, y coordinates)
- Current space/room
- Nearby users for proximity detection

Key Functionalities

- **Connection Handling:** When a user connects, they are assigned a unique ID and added to the active user list.
- **Movement Broadcasting:** User position updates are broadcast to all users within the same space.
- **Disconnection Handling:** When a user disconnects, they are removed from the active session list.
- **Event Processing:** All user actions (move, join, leave) are handled through event-based messaging.

2. User Movement and Sync Engine

The movement system is responsible for handling real-time updates of user positions across all clients.

Each movement event contains:

- User ID
- Current position (x, y)
- Direction of movement

The server processes this data and sends updates to all users in the same environment.

Movement Flow

1. User presses movement key
2. Client updates local position
3. Movement data sent to server
4. Server validates and broadcasts update
5. Other clients render updated position

This ensures a smooth and synchronized real-time experience.

3. Proximity Detection System

One of the key features of the platform is proximity-based interaction. Users can interact with others only when they are within a certain distance threshold.

A simple distance formula is used:

$$\text{distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Events Triggered

- **PROXIMITY_ENTER** → when users come close
- **PROXIMITY_LEAVE** → when users move apart

This system enables natural interaction similar to real-world environments.

4. Frontend Rendering > Next.js + Phaser

The frontend of the application is built using Next.js combined with Phaser for rendering the 2D virtual environment.

Rendering Components

- Tile-based map system
- User avatars
- Movement animations
- Interaction zones

Phaser is responsible for:

- Drawing the game canvas
- Updating player positions
- Rendering real-time changes

Next.js handles:

- Routing
- UI components
- API integration

5. Map and Environment System

The virtual world is structured into maps composed of grid-based tiles. Each tile represents a specific element such as:

- Walkable area
- Obstacle
- Interaction zone

Maps are stored in the database and loaded dynamically when a user enters a space.

Map Features

- Static elements (walls, objects)
- Dynamic elements (users, events)
- Layered rendering

6. Database Layer > Prisma + PostgreSQL

The backend uses PostgreSQL as the primary database, managed through Prisma.

Database Entities

- Users
- Spaces
- Rooms
- Maps
- Elements

Key Operations

- User authentication and storage
- Space creation and management
- Map data retrieval
- Session persistence

Prisma provides type-safe queries and simplifies database interactions.

7. API Layer (Backend Services)

The backend exposes REST APIs for managing application data.

Major APIs

- User authentication
- Create/join space

- Fetch map data
- Manage rooms

These APIs are consumed by the frontend and integrated with real-time events.

8. Authentication System

The system implements a basic authentication mechanism to manage user sessions.

Features

- User login/signup
- Session handling
- Token-based authentication

This ensures secure access to virtual spaces.

9. Data Flow

The overall system workflow follows a structured sequence:

1. User logs in and joins a space
2. Map data is fetched from the database
3. WebSocket connection is established
4. User movement events are sent to the server
5. Server broadcasts updates to all clients
6. Clients render updated positions in real time

10. Code Flow (High-Level)

The interaction between modules can be summarized as:

- **Frontend (Next.js + Phaser)** → Captures user input
- **WebSocket Server** → Processes real-time events
- **Database (PostgreSQL + Prisma)** → Stores persistent data
- **API Layer** → Handles structured data operations

11. Deployment Setup

The application is containerized using Docker for easy deployment and scalability.

Components Deployed

- Backend server
- WebSocket service
- Database (PostgreSQL container)
- Frontend application

12. Final Output to the User

The end-user interacts with an interactive virtual environment where:

- Users move freely in a 2D space
- Avatars represent participants
- Communication happens based on proximity
- Real-time updates ensure synchronized experience

CHAPTER – 3 : RESULTS AND DISCUSSION

The implemented system successfully demonstrates a real-time, interactive virtual environment where users can navigate, interact, and communicate dynamically. By integrating WebSocket-based communication with a game-rendered frontend, the system achieves low-latency synchronization across multiple clients.

The results validate the effectiveness of combining real-time networking with spatial interaction logic to simulate a collaborative digital space.

3.1 Real-Time User Interaction Output

The application allows multiple users to join a shared space and interact based on their positions. Each user is represented by an avatar that moves in real-time across the environment.

When a user performs movement actions, the updates are instantly reflected on all connected clients. This confirms the efficiency of the WebSocket-based event broadcasting mechanism.

Observed Results

- Smooth movement synchronization across users
- Minimal latency in position updates
- Accurate rendering of multiple users in a shared space

3.2 Proximity-Based Interaction Results

The proximity detection system was tested by moving users within and outside a defined interaction radius.

When two users come within the threshold distance:

- A **PROXIMITY_ENTER** event is triggered
- Interaction capability is enabled

When users move apart:

- A **PROXIMITY_LEAVE** event is triggered
- Interaction is disabled

Observations

- Accurate detection of nearby users
- No duplicate or inconsistent proximity events
- Stable interaction transitions

This validates the correctness of the distance-based logic implemented on the server side.

3.3 Frontend Rendering and Visualization

The frontend, built using Next.js and Phaser, successfully renders the virtual environment and user movements.

The system displays:

- Tile-based map layout
- User avatars
- Movement animations
- Real-time updates

The rendering pipeline ensures that visual updates occur instantly without flickering or lag.

Fig 3.1 – Virtual Environment and User Movement Interface

(Insert screenshot of Phaser canvas showing multiple users moving in the space)

The figure illustrates the main interface where users are visualized as avatars on a grid-based map. Each movement action is reflected immediately, demonstrating real-time synchronization.

3.4 Data Flow and Event Handling Output

The system's data flow was tested to ensure correct handling of events from input to visualization.

Execution Flow Observed

1. User input captured on frontend
2. Movement event sent via WebSocket
3. Server processes and updates state
4. Broadcast sent to all clients
5. Clients render updated positions

Results

- Event propagation is consistent
- No data loss during transmission
- Stable real-time updates under multiple users

Fig 3.2 – Real-Time Event Flow Execution

(Insert diagram/screenshot showing WebSocket message flow or logs)

This figure represents the communication between client and server, highlighting how movement events are processed and distributed.

3.5 Database Integration Results

The backend database successfully stores and retrieves application data such as users and space configurations.

Results Observed

- Fast retrieval of space/map data
- Reliable persistence of user sessions
- No data inconsistency issues

The use of structured schemas ensures data integrity and scalability.

Fig 3.3 – Database Structure and Stored Records

(Insert screenshot of database tables or Prisma schema visualization)

This figure shows how user and space data are organized and stored efficiently.

3.6 System Performance Analysis

The system was evaluated under multiple concurrent users to test performance.

Performance Metrics

- **Latency:** Low delay in movement synchronization
- **Scalability:** Handles multiple users within the same space
- **Responsiveness:** Immediate UI updates

Observations

- The system performs efficiently for small to medium-scale usage
- Slight delays may occur with a large number of users, indicating scope for optimization

3.7 Model Evaluation (If Applicable)

Although the system primarily focuses on real-time interaction, the logic components (like proximity detection and state management) were validated through consistent output behavior.

Validation Results

- Correct event triggering
- Stable state updates
- Accurate spatial calculations

3.8 User Experience Evaluation

The system provides an intuitive and engaging user experience.

Key Findings

- Easy navigation within the virtual space
- Natural interaction through proximity
- Real-time responsiveness enhances usability

Fig 3.4 – User Interaction and Proximity Event Trigger

(Insert screenshot showing two users interacting within proximity)

This figure demonstrates how interaction is enabled when users come close to each other in the virtual environment.

3.9 Limitations

Despite successful implementation, certain limitations were observed:

- Limited scalability for very large concurrent users
- Basic interaction features (can be extended further)
- No advanced physics or collision detection

3.10 Summary

The results confirm that the system effectively integrates real-time communication, spatial interaction, and frontend rendering into a cohesive platform.

The implemented architecture successfully:

- Enables synchronized multi-user interaction
- Provides proximity-based communication
- Delivers a smooth and responsive interface

Overall, the project achieves its objective of creating an interactive real-time environment and lays a strong foundation for future enhancements such as voice communication, AI-driven interactions, and large-scale deployment.

CHAPTER – 4 : CONCLUSION AND FUTURE SCOPE

4.1 CONCLUSION

The project titled “**2D Real-Time Metaverse Platform**” successfully demonstrates the design and implementation of a scalable, real-time spatial interaction system. The primary objective of developing an interactive virtual environment where users can move, communicate, and interact dynamically based on proximity has been effectively achieved.

The system integrates modern web technologies with real-time communication protocols to simulate a collaborative digital environment. By utilizing WebSocket-based architecture, the platform ensures low-latency synchronization of user movement and events, enabling seamless multi-user interaction. The incorporation of a structured environment consisting of spaces, rooms, and maps further enhances the modularity and extensibility of the system.

One of the key highlights of the project is the implementation of a **proximity-based interaction model**, which mimics real-world communication behavior. This approach allows users to engage naturally within the virtual space, creating a more immersive and intuitive experience compared to traditional communication platforms.

The frontend rendering using Next.js and Phaser provides a responsive and visually interactive interface, while the backend powered by PostgreSQL and Prisma ensures efficient data management and scalability. The system architecture effectively separates concerns between

real-time processing, persistent storage, and user interaction, resulting in a robust and maintainable design.

Performance evaluation indicates that the system performs efficiently for small to medium-scale concurrent users, with minimal latency and consistent synchronization. The implementation of event-driven communication ensures reliable propagation of updates across all connected clients.

Overall, the project successfully bridges the gap between static online communication systems and immersive virtual environments. It establishes a strong foundation for developing advanced metaverse applications and demonstrates the practical feasibility of real-time spatial interaction systems using modern web technologies.

4.2 FUTURE SCOPE

Although the system achieves its core objectives, there are several areas where it can be further enhanced to evolve into a fully-featured metaverse platform.

1. Integration of Real-Time Audio/Video Communication

The current system supports interaction based on proximity but does not include voice or video communication. Future development can integrate WebRTC-based audio and video streaming, allowing users to communicate naturally when they are within interaction range. This would significantly enhance immersion and usability.

2. Advanced Physics and Collision Detection

At present, the movement system does not enforce strict collision handling. Future improvements can include:

- Collision detection with map elements
- Pathfinding algorithms
- Physics-based movement

This will create a more realistic and controlled virtual environment.

3. Scalable Architecture Using Distributed Systems

To support a large number of concurrent users, the system can be scaled using:

- Load balancing
- Microservices architecture
- Distributed WebSocket servers
- Message brokers like Redis

This would enable the platform to handle thousands of users simultaneously.

4. 3D Environment Expansion

The current implementation is limited to a 2D environment. Future scope includes upgrading to a 3D metaverse using technologies like:

- Three.js
- WebGL
- Unity WebGL

This will provide a more immersive and visually rich experience.

5. AI-Based Interaction and Smart Avatars

Artificial Intelligence can be integrated to enhance user experience by:

- Smart NPC (Non-Playable Characters)
- AI-based recommendations
- Automated moderation
- Personalized virtual assistants

This would make the platform more interactive and intelligent.

6. Blockchain Integration for Digital Identity and Assets

Future development can include blockchain technology for:

- Secure user identity management
- Ownership of virtual assets
- NFT-based objects and spaces

This would enable a decentralized metaverse ecosystem.

7. Enhanced UI/UX and Customization

User experience can be improved by adding:

- Avatar customization
- Custom map creation tools
- Drag-and-drop environment builders
- Interactive UI dashboards

8. Mobile and Cross-Platform Support

Currently, the platform is optimized for web. Future enhancements can include:

- Mobile application support
- Cross-platform compatibility
- Progressive Web App (PWA) integration

9. Security Enhancements

Future versions can implement:

- End-to-end encryption for communication
- Advanced authentication mechanisms
- Role-based access control

10. Gamification and Social Features

Adding gamification elements can increase user engagement:

- Points and rewards system
- Virtual events and meetups
- Social networking features

4.3 FINAL REMARK

The project serves as a foundational step towards building next-generation virtual collaboration systems. With further enhancements and scalability improvements, it has the potential to evolve into a full-fledged metaverse platform capable of transforming how users interact, collaborate, and socialize in digital environments.